Phase 7 — Dashboard & Benchmark-Driven README

Goal of this phase: the presentation layer — the part that lands the project in 6 seconds with a recruiter and 20 minutes with an interviewer. A clean dashboard of the benchmark results and a README that's the centerpiece of the repo.

---

1. What I built

- `dashboard/build_dashboard.py` — generates a **self-contained** `dashboard/index.html` from `results/`: a summary table (throughput / p50 / p99 / TTFT per engine × concurrency), the four benchmark charts **inlined as base64** (so the file is portable and renders anywhere, including GitHub), and a quantization table that appears once the GPU study has run.
- **A rewritten README** as the project's front door: problem statement, a **Mermaid architecture diagram**, the three engine variants, headline benchmark results with the throughput graph embedded, key findings, a **Key Engineering Decisions** section, what I learned, and copy-paste run instructions (Python *and* one-command Docker Compose).
- Links from the README to every per-phase write-up and its interview-prep PDF.

Verification: dashboard generates from current results (6 benchmark rows); README renders with a working architecture diagram and the embedded throughput chart.

---

2. Design decisions

Decision	Why
Static generated HTML, not Streamlit	No server or extra runtime to run; a single file opens anywhere and can be hosted on GitHub Pages. Lower friction for a recruiter than "pip install streamlit && run".
Base64-inlined images	The dashboard is one self-contained file — no broken image links when shared or moved.
README leads with the result, then the method	Recruiters skim; the throughput graph and the flat-vs-scaling table are above the fold. Depth (decisions, learnings) follows for the interviewer.
Honest framing throughout	"The goal isn't to beat vLLM" + the TTFT-tradeoff callout signal engineering maturity, which reads better than inflated claims.
Per-phase PDFs linked	The write-ups double as my interview prep and as evidence of communication skill.

---

3. Why this phase matters (the part most candidates skip)

A strong project with a weak README underperforms a mediocre project with a great one, because the README is what actually gets read. Leading with measured results, explaining the *why* behind decisions, and being honest about tradeoffs is what turns "another LLM project" into a 20-minute technical conversation. The dashboard makes the numbers tangible at a glance.

---

4. Interview Q&A

Q: Walk me through this project in two minutes. A: I built an LLM inference engine three ways — a naive baseline, a from-scratch version with my own KV cache and continuous-batching scheduler, and a vLLM baseline — then benchmarked all three under load for throughput and p50/p90/p99 latency, studied the FP16/INT8 quantization tradeoff, and deployed on GKE with autoscaling. The naive server's throughput is flat with an exploding tail under load; my batched engine scales throughput and holds a lower tail; and I can explain exactly where vLLM's PagedAttention pulls ahead because I built the contiguous-cache version it improves on.

Q: What was the hardest part? A: The continuous-batching scheduler — specifically merging per-sequence KV caches of different lengths into one batched forward with correct RoPE positions and masking, while admitting and evicting requests mid-flight. Proving it with a test that batched output is byte-identical to single-sequence decoding.

Q: What would you do with more time / to make it production-grade? A: Adopt paged KV cache (or just run vLLM) to remove padding waste; add chunked prefill to protect TTFT under burst; scale GKE on a serving metric (queue depth / tokens-per-sec) instead of CPU; add streaming responses; and push to larger models where the batching and paging wins are far bigger.

Q: How do you know your numbers are trustworthy? A: Closed-loop load with explicit concurrency, client-side latency that includes queueing, percentiles rather than means, GPU-utilization sampling to confirm the GPU is actually used, and everything written to JSON/CSV and plotted — repeatable with one command.

---

5. One-line recall

> *"I made the results legible: a self-contained benchmark dashboard and a results-first README with an architecture diagram, the engine variants, headline graphs, the key findings, and an honest engineering-decisions section — so the project communicates its depth in seconds and holds up for twenty minutes."*